

RepoFlow: 仓库级代码智能体协作与健康评测

大语言模型与信息决策课程项目书面报告

骆春阳 陈奕好

2026 年 6 月

摘要: 代码智能体正从单轮代码补全转向仓库级任务执行。智能体可靠性不仅取决于模型能力，也受提示、工具、上下文、检查器和反馈回路等控制环境影响[1]。然而，在多轮顺序协作中，任务信息常保存在单个会话或模型上下文中，后续智能体难以继承任务边界、修改意图、验证证据和遗留风险，导致范围漂移、重复实现、交接缺失和维护性退化。对于真实软件工程场景而言，代码智能体不仅需要完成一次代码修改，还需要留下可被后续开发者和智能体理解的过程状态。

本文提出 RepoFlow，一种面向代码智能体顺序协作的仓库内控制框架。RepoFlow 将任务边界、看板状态、交接记录、验证证据和代码健康信号写入本地仓库，使后续智能体、脚本和人工复审者读取同一组事实。与依赖模型上下文窗口和在线工具编排的方式相比，仓库内共享状态可版本化、离线复现和差异审计。系统通过 task board、agent bridge、handoff 机制、GitHub Pull Request 接入和 health review 模块，将智能体执行过程转化为可追踪、可验证、可复审的仓库状态变化。这种设计并不替代底层模型能力，而是在模型之外提供一层轻量级的协作约束和复审基础设施，使任务推进过程不再只依赖智能体的自我声明，而能够由仓库文件、命令记录和检查结果共同支撑。

本文基于 Expense Lite 顺序任务链和若干开源 Python 仓库进行探索性测试。结果表明，RepoFlow 在小样本任务上不能被解释为提高正确率的通用方法；其主要价值在于将模糊失败拆解为正确性、范围漂移、交接缺失、证据不足和代码健康风险等可观察信号，从而提升协作中的诊断和复审效率。Health Review Skill 对比实验表明，与直接检查相比，skill-based review 更关注代码是否具有长期维护性，为后续智能体接手和人工复审提供更稳定的维护性参考。

关键词: 代码智能体；控制环境工程；仓库内共享状态；顺序协作；验收门控；代码健康评测

目录

1	引言	3
2	相关工作与理论背景	3
3	问题定义与失败模式	4
4	RepoFlow 方法设计	5
4.1	整体思想	5
4.2	从软提示到硬约束	5
4.3	主要模块	6
5	系统实现	6
5.1	关键文件树与任务生命周期	6
5.2	关键流程伪代码	6
5.3	代码健康审查	8
5.4	适应机制	9
6	实验设计与评价指标	10
6.1	任务链	10
6.2	实验条件	11
6.3	评价指标	11
7	实验结果与案例分析	12
7.1	课堂演示任务链	12
7.2	弱提示 B001 案例	12
7.3	Clean Restart B001-B002 案例	12
8	讨论与局限性	13
9	结论	13
A	AI 工具使用与贡献说明	14
A.1	项目材料来源、AI 工具使用与小组分工	14
B	项目材料与复现入口	14
C	大作业感想	14

1 引言

大语言模型驱动的代码智能体已经能够阅读仓库、修改多文件、运行测试并根据失败信息继续修复。此时，关键问题不再只是模型能否生成一段正确代码，而是生成行为能否进入可靠的软件协作流程。真实项目依赖任务看板、代码审查、持续集成和交接记录维持连续性；代码智能体一旦缺少这些外部状态，常会出现上下文断裂、范围漂移、证据缺失和维护性退化。

现有代码智能体产品通常把规则、工具和上下文组织在模型运行时周围。本文关注另一种互补路径：把顺序协作所需的最小事实写回仓库，使任务边界、交接记录、验证证据和代码健康信号成为可版本化对象。这样做的直接动机是，后续智能体往往无法访问前序智能体的私有对话，但它可以读取仓库；脚本无法判断模型是否“理解”任务，却可以检查文件差异、命令记录和交接模式；人工复审无法复原一次聊天全过程，却可以审查仓库中留下的证据。

本文提出 RepoFlow，一种面向代码智能体顺序协作的仓库内控制框架。它不替代底层模型，也不规定模型内部推理方式，而是在模型之外提供一组可读、可查、可复审的协作对象。本文的贡献包括：提出仓库内共享状态协议；设计结构化交接机制；构建代码健康审查模块；将正确性、范围控制、交接完整性、看板一致性和仓库卫生转化为可观察指标；并通过 Expense Lite 任务链展示该框架如何改变失败形态。本文的实验是课程项目规模的探索性案例研究，所有结论均限于这一证据边界。

2 相关工作与理论背景

代码智能体系统通常由模型、工具、上下文和外部控制逻辑共同构成。OpenAI 将这类模型之外的工程环境称为 harness，并强调规则、知识、工具、可观测性和反馈会共同塑造智能体行为[1]。Martin Fowler 网站上的讨论进一步把 harness 概括为围绕代码智能体建立的上下文、提示、工具、检查和人类反馈机制[2]。LangChain 等框架也将模型调用、工具选择和上下文管理组织为应用层运行时[4]。这些观点说明，代码智能体的可靠性不能只从模型参数或单次提示中理解。

Anthropic 对智能体系统的工程总结强调，许多有效系统并不是完全开放的自治体，而是由预定义 workflow、工具调用和反馈环节组成；在复杂度选择上，应优先采用简单、可组合、可调试的结构[3]。OpenCode、Codex CLI 和 Claude Code 等工具则代表了面向终端或仓库的代码智能体形态，用户可以让模型读取文件、执行命令并修改项目[5, 6, 7]。这类工具提升了单个开发者的自动化能力，但当任务需要跨轮次、跨智能体、跨上下文接力时，仅依赖一次会话中的模型记忆仍然不足。

多智能体框架提供了另一条路线。AutoGen 通过多智能体对话组织复杂任务[8]，LangGraph 和 CrewAI 等框架也提供图式流程、角色分工和多智能体编排能力[9, 10]。这些系统适合在一次流程中协调多个角色，但本文研究的并不是并发讨论、角色投票或集中调度，而是更接近日常软件工程的顺序接力：某一智能体完成一个任务后退出，下一智能体在同一仓库状态上继续工作。

基于上述背景，RepoFlow 与典型 harness 的区别在于控制事实的落点。典型 harness 更多依托运行时上下文、工具接口和模型周边编排；RepoFlow 则把关键约束沉淀到本地仓库中。二者并非替代关系，而是不同层面的控制手段。前者更灵活，适合快速配置工具和动态注入上下文；后者更适合版本控制、离线复现、差异审计和跨智能体交接。

从课程主题看，RepoFlow 属于智能体控制环境设计，而不是模型训练或单一提示优化。本文使用的关键术语如下：外层控制框架指模型之外用于约束、观察和验收智能体行为的环境；前馈引导指行动前提供规则、任务和边界；反馈传感器指行动后检测偏差或风险的检查机制；验收门控指决定任务能否完成并激活下一任务的外部判定；仓库内共享状态指由版本控制系统

维度	运行时 harness	仓库内共享状态
状态位置	会话上下文、工具层、平台配置	版本化文件、任务看板、交接记录、证据目录
主要优势	配置灵活，能快速适配不同模型和工具	可审计、可复现、可被后续智能体和脚本读取
主要风险	私有上下文难以复原，跨会话继承不稳定	需要维护规则文件，可能产生模式误报和流程成本
适用场景	单次复杂任务、动态工具调用、交互式协助	多阶段顺序协作、课程评测、需要留痕的工程流程

表 1: 运行时 harness 与仓库内共享状态的互补关系。

管理、可被后续智能体读取的协作事实。

3 问题定义与失败模式

本文研究顺序协作，而非并发多智能体开发。具体假设是：同一时间只有一个智能体拥有修改权；任务之间存在依赖顺序；不同阶段可以由不同智能体或不同模型接手；后续智能体不能依赖前序智能体的私有聊天历史，只能读取仓库中留下的文件、提交差异和验证证据。这一假设排除了并发合并、角色投票和冲突调度等变量，使问题集中在上下文继承、任务边界和交接纪律上。

形式化地说，给定初始仓库状态 R_0 、按依赖排列的任务 $T_1, \dots, T_n$ ，以及可能不同的代码智能体 $A_1, \dots, A_n$ ，顺序协作要求每个智能体在完成 T_i 后留下足够的仓库内状态，使 A_{i+1} 能够在不读取前序私有对话的情况下继续执行 T_{i+1} 。RepoFlow 关心的不是单次生成是否足够聪明，而是一次接力结束后，仓库是否保留了下一次接力所需的事实。图 1 展示了这种异构智能体顺序接力的基本形态：每个智能体退出时把共享状态写回仓库，下一智能体从同一仓库状态继续。

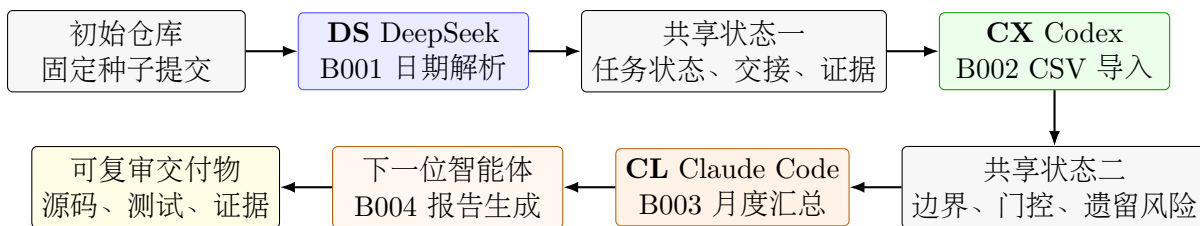


图 1: 异构代码智能体的顺序接力示意。图中使用颜色和缩写表示不同智能体，强调共享状态位于仓库而非私有对话。

在这种接力中，失败往往不只体现为代码不正确，而是体现为过程状态丢失。表 2 总结了顺序协作中五种典型的失败模式、具体表现，以及 RepoFlow 为每种模式引入的控制信号。

这些失败模式共同说明，代码智能体协作的关键不只是生成正确代码，还包括维护过程状态。一个代码修改即使最终通过测试，如果没有说明修改边界、验证过程和剩余风险，也会增加后续维护成本。相反，一个失败的修改如果能清楚暴露错误来源、越界文件和缺失证据，仍然对后续修复具有价值。

失败模式	表现	需要的控制信号
上下文断裂	后续智能体不知道前一阶段完成内容、遗留风险或下一步任务	任务看板、交接记录、上下文摘要
任务漂移	智能体提前实现后续任务，或重复实现已经完成的功能	当前任务贴纸、依赖状态、任务激活机制
范围漂移	修改超出当前任务允许路径，产生无关变更或格式化噪声	允许路径、基线提交、范围检查
证据缺失	声称测试通过但没有命令记录，或提前将任务标记为完成	验收命令、外部证据、看板一致性检查
代码健康退化	多轮修复留下临时函数、重复包装、调试文件或缓存残留	代码健康审查、仓库卫生检查、规则反馈

表 2: 代码智能体顺序协作中的主要失败模式。

4 RepoFlow 方法设计

4.1 整体思想

RepoFlow 将仓库视为代码智能体协作的事实来源。它把人类团队中常见的任务看板、工作协议、交接说明、验收证据和代码审查信号压缩为一组轻量级仓库文件和检查脚本。其设计原则是：可由智能体读取，可由脚本校验，可由人类复审，并且不依赖某一个特定模型或聊天平台。图 2 展示了 RepoFlow 的基本闭环：前馈引导让智能体在行动前知道当前任务和边界；智能体在工作区内修改代码并留下交接记录；反馈传感器检查行动结果；验收门控在工作区之外生成可信证据；只有门控通过后，下一任务才被激活。

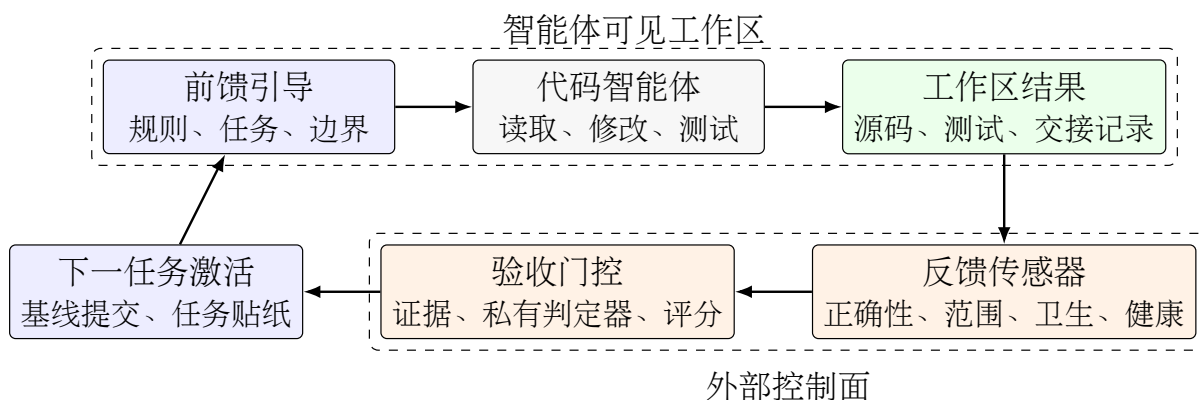


图 2: RepoFlow 的总体架构。智能体负责开放式代码修改，外部控制面负责检查、验收和任务激活。

这一设计刻意区分“智能体声称完成”和“系统接受完成”。

4.2 从软提示到硬约束

自然语言提示通常只能表达期望，难以保证执行。RepoFlow 的方法是把关键期望转化为可检查对象。例如，要求“不要乱改文件”会被转化为当前任务中的允许路径和基线提交，随后由范围检查器比较实际差异；要求“写清楚交接”会被转化为交接记录模式，随后由模式校验和证据校验检查；要求“不要提前完成任务”会被转化为看板状态规则，随后由外部验收门控

决定是否写入完成证据。图 3 概括了这一从自然语言期望到可执行约束的完整转换链条。

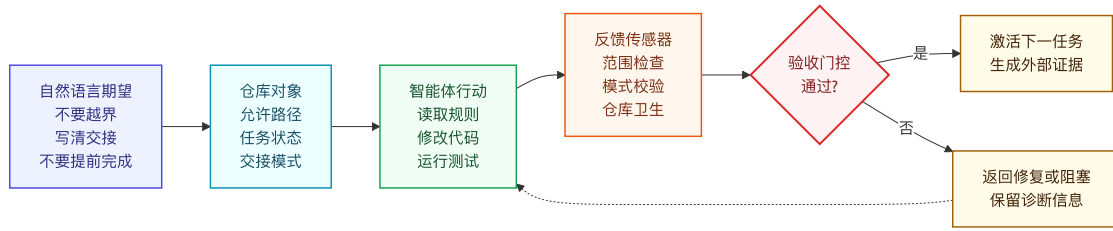


图 3: 从自然语言期望到可执行约束的转换链条。Mermaid 源文件随论文 TeX 一并保留。

行动前通过入口说明、行为合约和当前任务贴纸限定边界；行动中通过允许路径、验收命令和仓库卫生规则限制修改；行动后通过结构化交接、命令记录和风险字段保留证据；最终由外部评分、私有判定器和看板一致性检查决定任务是否可以进入下一阶段。通过把软提示固化为硬约束，RepoFlow 把“期望”变成了“可审计的失败信号”。

4.3 主要模块

RepoFlow 由五个相互配合的模块构成。BoardFlow 是任务状态协议，负责表示当前里程碑、任务依赖、状态、负责人、允许路径和当前交接记录。Agent Bridge 是过程管理层，负责刷新上下文、同步看板、写入交接记录和维护智能体可读摘要。Code Health Review 是维护性检查层，负责识别多轮智能体开发中常见的临时补丁、重复函数和可疑文件。Benchmark Control Plane 是实验控制层，负责初始化独立工作区、运行判定器、保存外部证据并激活下一任务。Independent Reviewer 是非阻塞复审层，负责从仓库差异和评分结果中报告语义风险，但不替代确定性门控。

这一模块划分避免了两个常见混淆。第一，RepoFlow 不是把所有规则写进一个提示词，而是将规则、状态、证据和复审分别落到不同层。第二，RepoFlow 不让智能体自己决定任务生命周期；智能体可以提出完成状态，但最终接受由外部控制面决定。

5 系统实现

5.1 关键文件树与任务生命周期

BoardFlowBench 是 RepoFlow 的实现与评测仓库。系统实现可以概括为：`.board/` 定义任务事实，`.repo_manager/` 提供智能体可读上下文，`repo_manager_core/` 执行检查和门控，`skills/` 暴露接手、交接和复审能力，`benchmark/` 与 `scripts/` 负责实验复现。图 4 给出了论文主线所需的关键文件树。

文件树体现了 RepoFlow 的“控制事实落点”原则：任务状态、规则策略、上下文摘要和证据目录都是仓库内的普通文件，智能体、脚本和人工复审者可以读取同一组事实。基于这些文件，代码智能体的单个任务生命周期如图 5 所示：智能体在行动前读取仓库内状态，行动中遵守任务边界，行动后留下结构化证据；任务是否完成则由外部门控判定。

5.2 关键流程伪代码

算法 1 描述了 RepoFlow 在完整任务链中的执行逻辑。它展示的不是某个具体 Python 函数，而是论文层面的系统行为：从固定种子提交初始化工作区，按依赖顺序暴露任务边界，注入协议文件，让智能体修改代码并留下交接记录，最后由外部门控决定是否进入下一任务。

算法 1 的核心是第 10 行的门控调用。验收门控是 RepoFlow 中最重要的边界：它将可执行检查和外部证据结合起来，防止智能体在工作区内部伪造完成状态。算法 2 给出了门控的内

```

BoardFlowBench/
|-- .board/
|   |-- tasks.yaml           # 机器可读任务看板
|   |-- handoff.schema.json  # 交接记录模式
|   '-- handoffs/*.json     # 每次接力的结构化交接记录
|-- .repo_manager/
|   |-- agent_context.md     # 下一位智能体可读的上下文摘要
|   |-- search_rules.json    # 代码健康扫描范围
|   '-- smell_rules.json     # 代码健康规则策略
|-- repo_manager_core/
|   |-- board/               # 看板、交接、范围、卫生和证据校验
|   |-- health/              # 函数扫描、代码味道检测、健康报告
|   '-- benchmark/           # 工作区、判定器、门控、结果聚合
|-- skills/
|   |-- agent-bridge/        # 任务接手和交接技能
|   |-- code-health-review/  # 代码健康审查技能
|   '-- boardflow-reviewer/  # 独立复审技能
|-- benchmark/
|   |-- targets/             # 目标仓库和固定种子提交
|   |-- tasks/expense_lite/  # B001 至 B004 顺序任务规格
|   '-- templates/           # 不同实验条件注入的协议模板
'-- scripts/
    |-- init_benchmark_workspace.py
    |-- run_scenario.py
    |-- finalize_benchmark_task.py
    '-- aggregate_benchmark_results.py

```

图 4: RepoFlow 实现中的关键文件树。图中只列出论文主线所需路径。

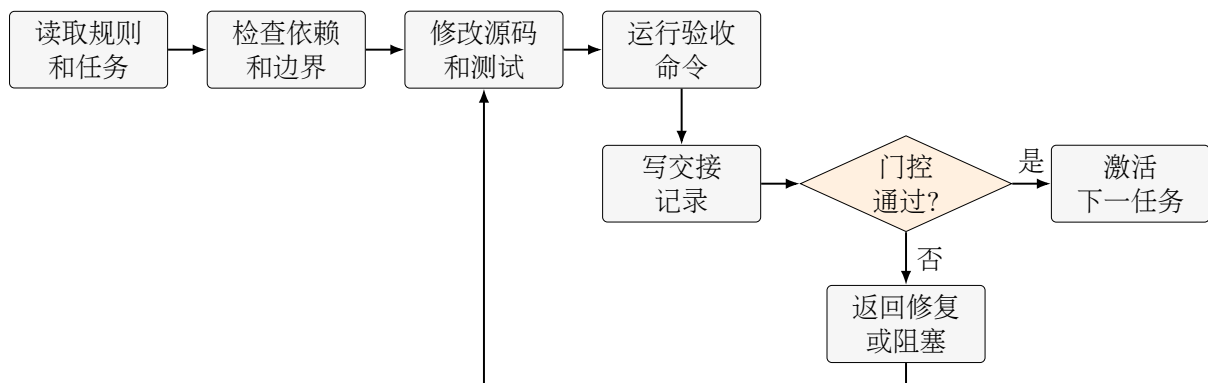


图 5: 代码智能体在 RepoFlow 中的任务生命周期。

Algorithm 1: Sequential Task Lifecycle

Input: Repository seed R_0 , ordered task list T , condition C

Output: Finalized checkpoints and evidence records

```
1:  $W \leftarrow \text{INITIALIZEWORKSPACE}(R_0)$ 
2:  $\text{INJECTPROTOCOLFILES}(W, C)$ 
3: for all  $t \in T$  in dependency order do
4:    $\text{EXPOSETASKBOUNDARY}(W, t)$ 
5:   if  $C = \text{full\_boardflow}$  then
6:      $\text{REFRESHAGENTCONTEXT}(W, t)$ 
7:   end if
8:    $\text{READREPOSITORYSTATE}(W)$ 
9:    $\text{EDITCODEANDTESTS}(W, t)$ 
10:   $\text{RECORDVALIDATIONANDHANDOFF}(W, t, C)$ 
11:   $s \leftarrow \text{FINALIZEGATE}(W, t, C)$ 
12:  if  $\neg s.\text{hardGatePass}$  then
13:     $\text{PRESERVEDIAGNOSTICEVIDENCE}(W, t, s)$ 
14:    return
15:  end if
16:   $\text{CREATECHECKPOINT}(W, t)$ 
17:   $\text{ACTIVATENEXTTASK}(W, T, t)$ 
18: end for
```

部检查顺序：先运行验收与回归检查，再检查允许路径和仓库卫生，只有在完整 BoardFlow 条件下才额外校验交接记录、看板一致性和不可信的完成状态。

Algorithm 2: Finalize Gate

Input: Workspace W , task t , condition C , external oracle O

Output: Score report s and gate decision

```
1:  $s \leftarrow \text{RUNACCEPTANCECHECKS}(W, t)$ 
2:  $\text{RUNREGRESSIONCHECKS}(W)$ 
3: if  $O \neq \emptyset$  then
4:    $\text{RUNEXTERNALORACLE}(O, W, t)$ 
5: end if
6:  $\text{CHECKALLOWEDPATHS}(W, t)$ 
7:  $\text{CHECKREPOSITORYHYGIENE}(W)$ 
8: if  $\text{REQUIRESBOARDFLOW}(C)$  then
9:    $\text{VALIDATEHANDOFFANDEVIDENCE}(W, t)$ 
10:   $\text{VERIFYBOARDTASKCONSISTENCY}(W, t)$ 
11:   $\text{REJECTUNTRUSTEDDONESTATE}(W, t)$ 
12: end if
13:  $\text{WRITESIGNEDEVIDENCE}(s)$ 
14: return  $s.\text{hardGatePass} \wedge \neg \text{HASBLOCKINGVIOLATION}(s)$ 
```

通过把门控逻辑从智能体行为中剥离出来，RepoFlow 保证了“完成”是一个由外部控制面签名的状态，而不是智能体自身写入的断言。

5.3 代码健康审查

Code Health Review 关注多轮智能体开发带来的维护性风险。它使用抽象语法树扫描函数定义、调用关系和文件结构，识别补丁式命名、未使用函数、薄包装函数、重复函数名、可疑目录和输出目录中的源代码文件。该模块不直接判定业务正确性，而是提供维护性风险的优先

Project	Mode	Input	Output	Cached Avg.	Δ Input
FastAPI	Direct inspection	2,810,793	5,991	73,105	–
FastAPI	Health Review Skill	933,647	4,407	33,166	66.8%
OpenWebUI	Direct inspection	543,176	5,105	40,192	–
OpenWebUI	Health Review Skill	477,935	4,458	28,880	12.0%
Superset	Direct inspection	685,888	10,098	36,522	–
Superset	Health Review Skill	334,837	2,951	26,688	51.2%
All projects	Direct inspection	4,039,857	21,194	49,940	–
All projects	Health Review Skill	1,746,419	11,816	29,578	56.8%

表 3: 直接让Agent检查与 Health Review Skill 的 Token 级对比。比较总输入和输出的tokens, Cached tokens 为每次运行的平均值。

检查方式	返回信号	解读
Skill-based view	default_parameter_value; duplicate_function_name; unused_function; wrapper_function; config_default_fallback; patch_name_smell.	输出稳定的仓库级 smell 分类: 过度默认参数、重复函数名、死函数、薄包装函数、隐式配置回退, 以及补丁/修复/临时/调试/安全式命名。
Direct inspection	Ruff lint: 2,482 个错误, 其中 1,860 个可自动修复; 遗留的 Optional[X]、Union[X,Y] 与 List[X] 注解; 178 个高复杂度函数; 过大的 middleware.py、config.py 与 main.py; 已禁用的 lint workflow lint-backend.disabled.	主要报告仓库能否通过 lint 或运行, 以及存在哪些明显的工程瓶颈, 但输出较随意, 跨仓库可比性较弱。

表 4: Health Review Skill 与智能体直接检查输出信号的定性对比。

级排序; 其扫描范围和命名策略由仓库内规则文件配置, 并允许通过人工反馈逐步校准。

为了检验该模块在大工程仓库中的实际作用, 我们在 FastAPI、OpenWebUI 和 Superset 三个 GitHub 开源项目上比较了两种检查方式: 一种是让智能体直接浏览仓库并给出代码健康判断, 另一种是先启用 Health Review Skill, 再让智能体围绕固定 smell taxonomy 进行审查。实验使用 kimi-2.7-code, 记录 input tokens、output tokens 和 cached tokens。表 3 给出了 token 层面的对比结果。

从成本角度看, Health Review Skill 将三仓库总 input tokens 从 4.04M 降到 1.75M, output tokens 从 21.2K 降到 11.8K, 平均 cached tokens 从 49.9K 降到 29.6K。更重要的是, skill 把审查任务从“仓库是否能运行”转化为“仓库是否存在可复现的维护性 smell”。表 4 总结了两种方式输出的差异: 直接检查更容易聚焦 lint、类型写法、复杂度和 CI 状态; Health Review Skill 则稳定输出仓库级 smell, 如未调用函数、重复函数名、薄包装函数、默认参数和补丁式命名。

5.4 适应机制

为避免健康审查过程停留在一次性的静态扫描, 本系统进一步引入了面向仓库和用户的适应机制, 使检查流程能够随着交互反馈不断调整。具体而言, 系统通过 smell_rules 与 search_rules 两类规则文件, 对代码异味识别和扫描范围进行动态配置。其中, smell_rules 主要用于记录哪些命名模式、函数结构或目录特征应被视为可疑、允许或需要结合上下文判断; search_rules 则用于控制检查范围, 例如是否排除测试目录、迁移文件、生成代码、第三方代码或框架约定入口。通过这种方式, 健康审查不再是固定规则下的机械扫描, 而能够根据不同仓库的技术栈、目录结构和框架习惯进行适配。

在交互过程中，用户可以对系统报告中的误报或重要信号进行反馈。例如，当某些函数被标记为未使用，但实际上属于 FastAPI 路由、pytest fixture、Alembic 迁移函数或插件回调时，用户可以将这类模式记录为允许项或上下文相关项。后续扫描中，系统会自动应用这些反馈规则，从而减少重复误报，提高报告的有效性。同样地，如果用户认为某些关键词，如 `temp`、`fix`、`helper`、`wrapper` 在特定仓库中具有特殊含义，也可以通过反馈机制调整其风险等级。

此外，系统还支持对仓库代码风格的持续学习。通过分析已有代码中的命名方式、目录组织、导入习惯、文档字符串风格、类型注解偏好和测试组织方式，系统可以生成并更新仓库级的风格画像。当 Coding Agent 在后续执行修改、重构或审查任务时，可以参考该风格画像，使生成的代码更符合当前仓库的既有规范。与此同时，用户在交互中表达的偏好，例如是否倾向于显式配置、是否接受薄包装函数、是否偏好较强的类型标注、是否要求更严格的模块边界，也可以被记录为长期偏好，用于后续任务的审查和生成。

因此，适应机制的核心作用在于将健康审查从“通用规则扫描”转变为“仓库感知、用户感知的持续治理过程”。系统不仅能够发现当前代码中的潜在问题，还能够在多轮交互中逐步理解项目的真实约定，区分框架模式与真正的代码异味，并形成可复用的长期记忆。这使得 RepoFlow 在面对不同规模、不同框架和不同开发风格的仓库时，能够提供更加稳定、个性化且可迭代的代码健康审查能力。

因此，Code Health Review 在 RepoFlow 中承担的是标准化审查层，而不是替代单元测试、lint 或运行时验证。它使后续智能体能够继承一组固定的 smell label，并在不同仓库之间比较相同类型的维护性风险。与 Agent 直接检查相比，Code Health Review 更关注代码长期来看是否健康，以及项目是否具有良好的维护性。

6 实验设计与评价指标

本文的实验定位为探索性案例研究。由于课程项目规模有限，实验样本不足以支撑统计显著结论，因此本文关注的是框架如何改变失败形态和诊断信息，而不是证明某种条件在所有任务上显著优于另一条件。实验围绕 Expense Lite 目标仓库展开，任务链、实验条件和评价指标分别如表 5、6 和 7 所示。

6.1 任务链

实验目标仓库为 Expense Lite，一个独立于 BoardFlowBench 的小型离线 Python 项目。每次实验从固定种子提交克隆工作区，避免智能体看到参考实现历史。任务链包含四个顺序任务，如表 5 所示。这四个任务从日期解析开始，逐步叠加 CSV 导入、月度汇总和报告生成能力，每一后续任务都依赖前面任务留下的接口和数据格式。

任务	名称	目标
B001	日期解析	支持斜杠日期并保持统一输出格式
B002	CSV 导入	从 CSV 读取费用记录并复用既有验证逻辑
B003	月度汇总	按月份和类别汇总金额并保持确定性排序
B004	报告生成	生成稳定 Markdown 报告并提供命令行入口

表 5: Expense Lite 顺序任务链。

这种递进式设计可以观察智能体在不同复杂度下的接力表现：B001 和 B002 偏重数据解析与导入，B003 和 B004 偏重汇总逻辑与命令行入口。随着任务推进，接口契约、输出格式和复用关系逐渐增多，为顺序协作中的上下文继承提供了测试场景。

6.2 实验条件

为了隔离 BoardFlow 各组件的贡献，BoardFlowBench 支持四种条件。本文重点讨论完整 BoardFlow 条件与无看板基线，同时保留中间条件作为系统设计的一部分。表 6 给出了各条件的具体差异。

条件	说明
no_board_baseline	不注入 BoardFlow 文件，智能体只面对普通代码仓库和任务提示。
native_instructions	只注入所选智能体原生入口说明文件。
native_docs_handoff	注入自然语言项目板和交接文档，但不注入机器可读 .board/。
full_boardflow	注入完整任务看板、结构化交接、上下文刷新、验收门控和任务激活机制。

表 6: 实验条件。

条件设计遵循“逐步增加控制面”的思路：无看板基线只保留原始任务提示；`native_instructions` 注入所选智能体原生入口说明；`native_docs_handoff` 增加自然语言项目板和交接文档；`full_boardflow` 再叠加机器可读看板、结构化交接和验收门控。通过比较这四个条件，可以大致区分“提示增强”“文档增强”和“硬约束增强”各自带来的过程差异。

6.3 评价指标

评价指标分为五类，覆盖从功能正确性到维护性风险的多个维度。正确性衡量任务功能是否满足可执行测试和私有判定器。范围控制衡量修改是否落在允许路径内。交接完整性衡量交接记录是否存在、模式合法并包含真实命令和测试证据。看板一致性衡量人类可读看板与机器可读任务状态是否一致。仓库卫生和代码健康衡量缓存、临时文件、可疑命名和维护性风险。表 7 汇总了各类指标的可观察信号和解释边界。

指标类别	可观察信号	解释边界
正确性	验收命令、回归命令、私有判定器	判断功能是否满足任务要求
范围控制	基线提交与实际差异、允许路径	判断是否发生越界修改
交接完整性	交接记录模式、命令与测试字段	判断接力信息是否可复审
看板一致性	任务状态、依赖、完成证据镜像	判断状态是否被错误推进
仓库卫生与健康	缓存、临时文件、可疑函数和文件结构	判断多轮开发是否损害维护性

表 7: 评价指标。

这些指标的共同特点是“可观察、可脚本化、可审计”。它们不依赖模型内部状态，只依赖仓库中留下的文件、命令记录和判定器输出，因此适合在不同智能体和不同运行之间进行比较。

7 实验结果与案例分析

本节通过三组观察展示 RepoFlow 如何改变失败形态：一次完整的课堂演示任务链、一个弱提示下的 B001 案例，以及一次 clean restart 的 B001-B002 对比。

7.1 课堂演示任务链

在一次 B001 至 B004 的课堂演示中，完整 BoardFlow 条件和无看板基线都完成了四个任务，任务通过率均为 1.00。这个结果说明，在较小且提示较明确的任务链上，RepoFlow 不一定直接提高最终正确性。差异主要体现在过程轨迹：完整 BoardFlow 条件产生了任务状态更新、结构化交接记录、门控证据和任务激活记录；无看板基线主要留下代码差异，缺少可由后续智能体直接继承的过程状态。表 8 给出了聚合结果。

条件	运行数	阶段数	通过率	范围漂移	交接违规
full_boardflow	1	4	1.00	0	0
no_board_base1 ine	1	4	1.00	0	不适用

表 8：课堂演示 B001 至 B004 聚合结果。

因此，课堂演示的结论不应写成“RepoFlow 提高正确率”，而应写成“RepoFlow 增加了过程可观察性”。在 B002、B003 和 B004 的门控修复中，隐藏契约、公共接口名称、命令行形状、输出路径和报告格式都可能导致失败。完整 BoardFlow 条件能够把这些失败与交接记录和看板状态关联起来，使后续修复更容易定位。

7.2 弱提示 B001 案例

为了测试控制面在提示变弱时的表现，我们单独运行了弱提示 B001 实验。如表 9 所示，完整 BoardFlow 条件正确识别当前任务，并保持范围和仓库卫生干净，但交接记录包含一个不被模式允许的字段，导致硬门控失败。无看板基线同样通过了 B001 的功能判定，却推断当前进度为 CSV 导入任务，创建了越界数据文件并留下缓存残留。

条件	正确性	硬门控	主要失败
full_boardflow	通过	失败	交接记录含不被模式允许的字段，任务范围和仓库卫生保持干净
no_board_base1 ine	通过	失败	提前跳到后续 CSV 任务，创建越界数据文件并留下缓存残留

表 9：弱提示 B001 实验结果。

该案例说明，RepoFlow 并不保证智能体永远满足流程格式，但它能把流程错误暴露为具体的模式违规；无看板条件则更容易把任务误判和范围漂移隐藏在“功能似乎可用”的结果中。

7.3 Clean Restart B001-B002 案例

在 clean restart 对比中，两种条件均通过 B001，但均在 B002 失败。共同业务错误是 CSV 导入没有复用既有验证逻辑，导致日期未规范化、文本空白未清理、金额仍为字符串。该结果表明，外层控制框架不能替代清晰需求和充分测试，也不能保证模型自动理解隐式业务契约。表 10 对比了两种条件在多个维度上的差异。

然而，两种条件的失败形态不同。完整 BoardFlow 条件除业务错误外，还暴露了越界数据

维度	完整 BoardFlow	无看板基线
B001 结果	通过，含完整交接与外部证据	通过，无任务看板和交接要求
B002 正确性	失败，未复用验证逻辑	失败，未复用验证逻辑
范围控制	检出越界 CSV fixture 文件	主要体现为代码级失败
交接状态	存在交接记录但模式不合法	不适用，缺少交接记录
完成证据	检出提前完成且证据镜像不存在	无任务状态可审计
诊断质量	能定位为业务、范围、交接和证据多类问题	主要依赖代码差异和判定器输出

表 10: Clean restart B001-B002 案例对比。

文件、交接记录字段缺失和提前完成状态缺少可信证据等流程问题。无看板基线主要暴露为代码和判定器失败，工作区本身较少说明智能体运行过哪些命令、为何没有补充测试以及下一步应如何修复。这些案例支持一个谨慎结论：RepoFlow 的主要收益不是让弱提示下的智能体必然一次写对业务逻辑，而是让失败更结构化、更可审计、更适合后续修复。

8 讨论与局限性

RepoFlow 的价值在于把代码智能体协作中的隐性过程转化为仓库内可观察状态。它适用于多阶段任务、多人或多智能体接力、需要复审证据的开发流程。特别是在后续智能体不能访问前序私有对话时，仓库内共享状态能够提供最低限度的上下文连续性。

但本文实验仍有明显局限。第一，样本规模较小，现有结果只能作为探索性案例，不能支持统计显著结论。第二，Expense Lite 是离线小型 Python 项目，不能代表大型前端系统、多语言仓库、机器学习训练工程或机器人项目。第三，RepoFlow 引入了额外文件和流程成本，小任务中可能看不到明显收益。第四，Health Review 的检查机制仍较依赖预定义搜索规则和启发式判断，存在误报风险。第五，适应机制仍需进一步泛化，使系统能够在与用户的持续交互中逐步提升检查质量。

未来工作应优先扩大实验规模，并将结论从“案例现象”推进到“可重复比较”。短期可以增加重复运行次数、智能体种类和提示质量条件。中期可以将代码健康规则与真实审查反馈结合，减少误报并建立更稳定的维护性指标。长期可以把 RepoFlow 做成较为成熟的工具，配备界面和实用工具以及反馈处理等集成功能。

9 结论

本文提出 RepoFlow，一种面向代码智能体顺序协作的仓库内控制框架。该框架的核心观点是：当代码智能体进入真实软件开发流程后，工程重点不能只放在模型能力上，还必须设计可被智能体读取、可被程序校验、可被人类复审的协作环境。RepoFlow 通过 BoardFlow 协议、Agent Bridge、Code Health Review、Benchmark Control Plane 和独立复审机制，将任务状态、交接记录、验证证据、范围控制和代码健康评测组织为一个闭环。

RepoFlow 不应被解释为提高所有任务正确率的通用方法。它更准确的作用是改变失败形态：将模糊的“智能体失败”拆解为正确性、范围、交接、看板一致性、仓库卫生和代码健康等可观察信号。在长程项目中，RepoFlow 有望进一步减少风格漂移、冗余代码和上下文记忆丢失等问题，并提升代码智能体协作过程的可维护性。

A AI 工具使用与贡献说明

A.1 项目材料来源、AI 工具使用与小组分工

本项目的研究问题、协议设计、仓库实现、实验脚本、PPT 初稿与演示材料，均来自小组围绕课程主题开展的项目开发过程。代码智能体在本项目中具有双重角色：一方面，它是本文的研究对象，即 RepoFlow 关注代码智能体如何在共享仓库环境中进行任务分解、顺序协作、状态交接与健康审查；另一方面，它也是项目实现和材料整理过程中的辅助工具，用于提升代码框架搭建、实验整理和文档撰写的效率。

在 AI 工具使用方面，本项目主要使用代码智能体完成文件检索、仓库结构理解、代码整体框架和模板生成、实验脚本辅助编写、实验测试过程整理、实验报告初版生成以及 LaTeX 排版优化等工作。核心算法及结构设计均为人为修改和编写。

本项目的小组分工如下：骆春阳主要负责 `health_review` 模块的设计与实现，包括代码健康审查流程、代码异味检测规则、自适应机制模块、实验测试与结果分析；同时负责后续 UI 界面设计、PPT 制作、演示材料整理以及书面报告的整体撰写与排版。陈奕好主要负责 `task.board`、`agent_bridge` 与 `handoff` 机制的设计和实现，包括任务板管理、智能体执行接口、任务交接协议，以及接入 GitHub 层面的 Pull Request 协作流程；同时参与书面报告中系统协议、协作流程和实现细节相关部分的撰写。

B 项目材料与复现入口

源代码仓库：<https://github.com/ChenNeuro/BoardFlowBench.git>。

目标仓库：<https://github.com/ChenNeuro/ExpenseLiteBenchDemo.git>。

关键复现入口包括 `scripts/init_benchmark_workspace.py`、`scripts/run_scenario.py`、`scripts/finalize_benchmark_task.py` 和 `scripts/aggregate_benchmark_results.py`。实验任务规格位于 `benchmark/tasks/expense_lite/`，实验条件位于 `benchmark/scenarios/`，案例记录位于 `docs/`。

Health_Review部分的测试入口：

- Apache Superset: <https://github.com/apache/superset>。该项目是一个大型数据可视化与商业智能平台，用于测试 RepoFlow 在大型 Python 仓库中的健康审查能力。
- Open WebUI: <https://github.com/open-webui/open-webui>。该项目是一个面向大语言模型应用的 Web 平台，用于测试 RepoFlow 在 AI 应用型仓库中的代码结构分析与健康审查能力。
- Full Stack FastAPI Template: <https://github.com/fastapi/full-stack-fastapi-template>。该项目是一个基于 FastAPI 的全栈项目模板，用于测试 RepoFlow 在中小型现代 Python/FastAPI 项目中的构建健康、代码结构和审查结果差异。

C 大作业感想

骆春阳：

这次大作业也进一步激发我对 Agent 方向的兴趣。在完成 RepoFlow 项目的过程中，我逐渐认识到，代码智能体并不是简单地让模型自动写代码，而是涉及任务分解、上下文传递、工具调用、状态记录、交接机制、代码审查和长期维护等一系列问题。相比单纯追求某一次生成结果是否正确，我现在更加关注如何设计外部控制机制，让智能体的行为变得可观

察、可复审、可持续改进。

这次大作业只是一个起点。RepoFlow 目前仍然是课程项目规模的原型，很多设计还不够成熟，但正是通过这些不完善之处，我更加清楚地看到了 Agent 系统落地中的真实问题。后续我会继续围绕代码智能体、仓库级协作、代码健康审查和长期记忆机制做更多学习和调研，也希望将这个项目持续推进下去，使其从一个课程项目逐步发展为更完整、更实用的 Agent 协作框架。

最后，非常感谢老师一学期以来的教学与指导。课程不仅帮助我建立了对大模型和智能体系统的整体认识，也让我通过大作业真正进入了一个自己感兴趣的研究方向。这个项目让我意识到，Agent 不是一个单纯的工具概念，而是连接模型能力、软件工程和人机协作的重要方向。未来无论是在科研还是实际开发中，我都会继续沿着这一方向探索下去。

陈奕好：通过本次课程项目，我不仅参与完成了 RepoFlow 的协议设计、顺序智能体交接、验收门控、代码健康审查和实验评测，也经历了从问题提出、系统实现到论文写作与成果展示的完整科研过程。实践使我认识到，大模型能力并不只取决于模型本身，规范的任务边界、共享状态和可验证证据同样重要。更有意义的是，我们开发的 Agent Bridge、Code Health Review 等技能反过来辅助了论文整理、实验复现和仓库维护，使项目工具真正服务于自身科研。我因此加深了对智能体工程、软件协作和实证研究方法的理解，收获颇丰。

参考文献

- [1] OpenAI. Harness engineering[EB/OL]. <https://openai.com/index/harness-engineering/>.
- [2] Böckeler B. Harness engineering for coding agent users[EB/OL]. Martin Fowler, 2026-04-02. <https://martinfowler.com/articles/harness-engineering.html>.
- [3] Erik S., Zhang B. Building effective agents[EB/OL]. Anthropic, 2024-12-19. <https://www.anthropic.com/research/building-effective-agents>.
- [4] LangChain. Agents documentation[EB/OL]. <https://docs.langchain.com/oss/python/langchain/agents>.
- [5] OpenCode. Documentation[EB/OL]. <https://opencode.ai/docs/>.
- [6] OpenAI. Codex CLI[Z/OL]. <https://github.com/openai/codex>.
- [7] Anthropic. Claude Code overview[EB/OL]. <https://code.claude.com/docs/en/overview>.
- [8] Microsoft. AutoGen documentation[EB/OL]. <https://microsoft.github.io/autogen/stable/>.
- [9] LangChain. Multi-agent systems in LangGraph[EB/OL]. https://langchain-ai.github.io/langgraph/concepts/multi_agent/.
- [10] CrewAI. Documentation[EB/OL]. <https://docs.crewai.com/>.